expin

Take-Home Task: Merchant Campaign Performance Widget

Expin — Product Engineer (Merchant Web)

Thanks for getting this far. This is the task we use to judge fit — output first, polish second. Spend 4–8 hours of focused time on it.

This role is focused on merchant-facing web features: dashboards, campaign management flows, performance views, and operational tools. React Native familiarity is a plus, but this task evaluates web execution specifically.

All data and code in this task are self-contained on your machine. The starter includes a local mock API server. There is no connection to any Expin infrastructure, codebase, or data.

Getting the starter

Download the starter zip from:
https://drive.google.com/drive/folders/1NgCNjZlz1mD8oKRMYa5Ozn58cpJ1Ety6?usp=sharing

Unzip it, then follow the README inside to push it to your own GitHub repo and run the project locally. Total setup is about 5 minutes.

What you are building

A Campaign Performance widget for a merchant dashboard.

A merchant has launched a campaign with 20–100 creators participating. Your widget shows live performance per creator and lets the merchant take action on underperformers.

Requirements

1. Creator performance table

Show each creator with: name, post status (pending / live / completed), views, conversions, conversion rate, and a Boost button. Default sort by conversion rate descending.

2. Filter bar

Filters: post status (multi-select), minimum conversion rate (slider), search by creator name.

Filter state lives in URL query params, so a merchant can share a filtered view with their team.

3. Live updates

Performance numbers change every 2–5 seconds via the mock stream endpoint included in the starter. The UI updates smoothly — no janky full re-renders.

4. Boost action

expin

Clicking Boost optimistically marks the creator as boosted, fires the boost API call (mock endpoint randomly succeeds or fails after 1–3s latency), and reverts on failure with a toast notification.

5. Empty, loading, and error states

Every surface — table, filter results, boost action — should handle these cleanly.

6. Arabic + English support

Toggle in the UI. Layout must mirror correctly in RTL. Real RTL parity, not faked.

Stack

The starter is pre-configured with:

- React 18 + Vite + TypeScript
- TanStack Query
- Zustand
- Shadcn/ui
- Tailwind CSS
- A local mock API server (Node + Express) with three endpoints: GET /creators, GET /stream (SSE), POST /creators/:id/boost
- Seed data: 50 fake creators with realistic-looking metrics, generated locally

The README in the starter has the full setup commands.

Submission

Reply to this email with all four items:

1. **1. GitHub repo link.** Public, or private with our reviewer invited as a collaborator (we will send the username separately).
2. **2. Commit SHA.** The specific commit you want us to grade against. We evaluate that commit, not whatever HEAD becomes later.
3. **3. Loom link.** 60–120 seconds walking through the UI and your three most interesting decisions. Show the widget running on your machine; explain the reasoning, not just what it does.
4. **4. AI workflow evidence.** In the repo, include the actual artifacts you used: a CLAUDE.md or .cursorrules file if you wrote one, the specs you fed to your agent, or the prompts you reused. We want the real files, not a description of how you used AI.

---

expin

Time

Spend 4–8 hours of real focused time. We would rather see well-thought-out partial completion than rushed-everything. If you cannot finish, finish what is most important and note what you would do next in the README.

What we are evaluating

- Does it work, and does it feel good to use?
- State architecture: server state vs client state, URL state, optimistic updates, race conditions handled cleanly.
- RTL parity: does Arabic actually work, or did you fake it?
- Code quality: readable on first scan, PRs we would merge.
- AI workflow: are you AI-first or AI-occasional?
- Communication: the Loom and AI artifacts tell us how you think.

Live walkthrough

If we move forward after your submission, the next step is a 30-minute call. On that call, we may ask you to modify your submission live or explain specific code decisions in detail.

Do not submit code you cannot explain. We are happy to hire engineers who used AI for most of the work — we are not happy to hire engineers who do not understand the work that came out.

Compensation and timeline

- AED 500 paid on submission, for submissions that make a genuine attempt, include all four required items, and run locally from the submitted commit. Broken, empty, or non-runnable submissions do not qualify.
- We aim to give you a decision within 48 hours of submission.
- If we move forward, the next step is the 30-minute call described above.

Ground rules

- Use any AI tools you want. That is the point of the role.
- Use any open-source libraries you want, on top of what the starter already includes.
- Do not pull existing code from other projects you have built. We want to see how you ship this one.

expin

- Questions are welcome — reply to this email and we will get back to you within a few hours.

Good luck. We are looking forward to seeing what you ship.

— The team at Expin